

DelosDB

Final Architecture & Implementation Proposal

One SQL Frontend · N2O MVCC · DelosPageVolume · Derby-Native Code Generation
L4 Cost Model · Capability-Segregated Contracts · Separation of Concern

Sources: PostgreSQL · MariaDB/InnoDB · H2 · Apache Calcite · HerdDB · MapDB · LeanStore · Umbra
Research: Wu et al. VLDB 2017 · Alhomssi & Leis VLDB 2023 · Jasny PVLDB 2026 · Graefe 1995 · Soliman SIGMOD 2014

June 2026 · DelosDB Engineering · Confidential

VLDB 2026, Graefe 1995, Soliman et al. SIGMOD 2014), and the JDK 21–25 platform roadmap. Every architectural decision is stated with its evidence and its

PART 1

Architecture — Four Independent Layers

Verified from PostgreSQL, MariaDB, Calcite, HerdDB, MapDB source

The architecture separates DelosDB into four layers. Each depends only on the layer below it through a narrow interface. No layer imports implementation classes from another layer. This is the architectural discipline that distinguishes every successful system studied: InnoDB's clean separation of undo log from storage, Calcite's capability-segregated Table interfaces, MapDB's Volume hierarchy. MariaDB's 218-method handler and PostgreSQL's `t_xmin` baked into the physical tuple are the counter-examples.

SQL EXECUTION LAYER — ONE frontend for ALL providers

- Derby parser (JavaCC) · Derby binder · Derby OptimizerImpl · AsmJava code generator
- GenericStatement pipeline: `parse` → `bind` → `optimizeStatement` → `generate`
- SAME JVM bytecode for heap and `delos_mvcc`: `INVOKEINTERFACE ResultSetFactory.getTableScanResultSet(22 args)`
- Runtime dispatch at `GenericResultSetFactory` — no compile-time provider branch
- `DelosTableScanResultSet` · `DelosInsertResultSet` · `DelosDeleteResultSet` · `DelosUpdateResultSet`
- `DelosVersionedStorageQueryTreeClassifier` (JavaCC parse-tree, no regex)

Depends on: Contract Layer interfaces only

CONTRACT LAYER — Capability-segregated interfaces (Calcite `FilterableTable` lesson)

- `DelosTableAccess` · `DelosFilterableTableAccess` · `DelosIndexableTableAccess`
- `DelosMutableTableAccess` · `DelosCostableTableAccess`
- `DelosTableCapability` (structural) · `DelosTableGuarantee` (semantic promises)
- `DelosRowIdentity` (opaque — no cross-provider equality, no physical address exposure)
- `DelosAccessContext` (typed key-lookup, Derby `ContextManager` idiom)
- `DelosPredicate` · `DelosScan` · `DelosProjection` · `DelosMutationPreparation`

Depends on: MVCC Layer interfaces only

MVCC CONCURRENCY LAYER — N2O Main+Delta, MV-SSI, no lock manager

- `MvccSnapshot` · `MvccCapturedVisibility` · `MvccStatementSnapshot` · `MvccCommandSequence`
- `MvccVersionChain` (`ReentrantReadWriteLock`) · `MvccVersionStore` (in-memory N2O old versions)
- `MvccTransactionRegistry` (volatile `TransactionSnapshot` — lock-free read path)
- `MvccLazyScan` (key snapshot at open, lazy value reads — no full materialization)
- `MvccVacuumWatermark` · `MvccSnapshotLease` · `MvccVacuumEngine`
- `MvccPageMutationLog` · `MvccTransactionOutcomeLog` · `MvccPageRecoveryRunner`

Depends on: I/O Layer interfaces only

I/O LAYER — DelosPageVolume (HerdDB ManagedFile + MapDB Volume + JDK 22 FFM)

- DelosPageVolume interface: 6 methods, SyncPolicy enum, ReentrantLock throughout
- FileChannelPageVolume · MappedPageVolume (MemorySegment+Arena, JDK 22)
- OffHeapPageVolume (MemorySegment[] O(1) array, not TreeMap) · FaultInjectingPageVolume
- ODirectPageVolume (HerdDB ExtendedOpenOption.DIRECT, Apache 2.0) — future
- IoUringPageVolume (FFM Linker + liburing, JDK 22+) — future
- DelosVolumeFactory: matched pairs (page + mutation-log + outcome-log + index)

Depends on: JDK APIs only — zero DelosDB imports

PART 2**Code Generation — One Frontend, Runtime Dispatch**

Verified from FromBaseTable.java, GenericResultSetFactory.java, ActivationClassBuilder.java

This is the foundational design fact of DelosDB's SQL execution. The same JVM bytecode is generated for a heap table and a delos_mvcc table. The provider identity is determined at runtime, not at compile time. Two SQL frontends is architecturally incoherent: a query spanning heap and delos_mvcc tables in a JOIN produces one QueryTreeNode tree, one FromList, one OptimizerImpl call, and one generated activation class. Splitting at SQL parse time is impossible; splitting at bind time breaks join planning. The GenericResultSetFactory dispatch is the correct and only boundary.

```

COMPILE TIME — FromBaseTable.generateResultSet():
    acb.pushGetResultSetFactoryExpression(mb)           // this.getResultSetFactory()
    mb.callMethod(INVOKEINTERFACE, null,
        "getTableScanResultSet",                       // SAME method name for ALL providers
        NoPutResultSet, 22_args)                      // conglomerId baked in — IGNORED by delos_mvcc

RUNTIME — GenericResultSetFactory.getTableScanResultSet():
    DelosTableScanProviderLookup.find(activation, tableName)
    → DataDictionary.getTableDescriptor(tableName)
    → SYSTABLES.STORAGEPROVIDER = "delos_mvcc" ?
        YES → DelosTableScanResultSet.createIfEnabled(params)
        NO  → new TableScanResultSet(params)           // Derby heap path unchanged

```

Implication for optimizer design: because the generated bytecode is identical, any optimizer improvement must work within Derby's single compilation pipeline. The correct intervention for delos_mvcc tables is L4 — wiring EngineMvccTableAccess.estimateTableCost() into Derby's cost model at the point where the optimizer already calls DelosNativeTableCostLookup.observeIfEnabled() inside FromBaseTable.optimize(). The provider identity is already visible to the optimizer via tableDescriptor.getStorageProviderName(). No preprocessing layer, no rule engine, no second SQL path is needed or correct.

PART 3**MVCC Design — N2O Main+Delta, MV-SSI**

Wu et al. VLDB 2017 · Alhomssi & Leis VLDB 2023 · Pavlo CMU 2023

Decision	Choice	Evidence
Concurrency control	MV-SSI (Snapshot Isolation) No lock manager	MvccSnapshot + MvccCapturedVisibility already implemented. Write conflicts at MvccWriteConflictException. Avoids 2PL (InnoDB) and OCC validation (Hekaton).
Version storage	N2O: MvccPageFile (current) + MvccVersionStore (old versioning)	InnoDB model: current in clustered index, old in undo log. Delta: version chains in memory, associated with pages. NOT PostgreSQL O2N (bloat, chain traversal, full-row copy).
Garbage collection	Tuple-level background vacuum with watermark pinning	MvccVacuumWatermark + MvccSnapshotLease already designed. Vacuum touches MvccVersionStore only — MvccPageFile never bloats. MvccHistoryPrunedException distinguishes pruned from invisible.

Decision	Choice	Evidence
Index management	Logical pointers (row key, not physical page offset)	DelosMvccIndex: NavigableMap<Object, LinkedHashSet<K>>. Survives physical row movement and version pruning. Consistent with InnoDB, Oracle, NuoDB, HyPer.

Three structural changes required

Version storage split

MvccTable currently holds ALL versions (current + historical) in one LinkedHashMap. Split: MvccPageFile holds current committed version per row (non-MVCC read = $O(1)$). MvccVersionStore holds N2O chain of old versions (MVCC read traverses only when needed). Vacuum prunes MvccVersionStore — MvccPageFile never accumulates dead versions.

Concurrency — lock-free read path

MvccTransactionManager has 15 synchronized methods on one monitor. Fix: volatile TransactionSnapshot published atomically on every commit/abort. statusOf() / commitSequenceOf() become lock-free reads of the immutable snapshot. MvccVersionChain: synchronized → ReentrantReadWriteLock (multiple concurrent readers, exclusive only for mutation). Follows MapDB ByteBufferVol.growLock pattern (Apache 2.0).

Lazy scan cursor

MvccTable.openScan() materializes the entire visible result set before returning. Fix: MvccLazyScan takes a key-set snapshot at open time (fast), then advances lazily per next() call: check MvccPageFile current version → if not visible, check MvccVersionStore. Table lock not held between next() calls. MVCC's fundamental promise — readers never block writers — delivered for the first time.

Recovery contract

On database open (MvccPageRecoveryRunner):

1. Read MvccTransactionOutcomeLog
 - COMMITTED transactions → rehydrate MvccTransactionRegistry
 - ABORTED transactions → mark aborted in registry
 - UNRESOLVED transactions → abort-on-unresolved (safest default)
 2. Apply MvccPageMutationLog to MvccPageFile
 - COMMITTED entries: write current version to MvccPageFile
 - ABORTED entries: discard (never committed)
 3. Rebuild MvccVersionStore from MvccPageFile version chain
 4. Publish initial TransactionSnapshot → accept new transactions
- GAP today: step 1 and 4 not yet wired. MvccTransactionOutcomeLog exists.
MvccPageRecoveryRunner exists. Integration is the remaining work.

PART 4

I/O Layer — DelosPageVolume

HerdDB Apache 2.0 · MapDB Apache 2.0 · JDK 22 JEP 454 · Leis VLDB 2024

```
public interface DelosPageVolume extends AutoCloseable {
    MvccPage readPage(MvccPageId id) throws IOException; // O(1) by page id
    void writePage(MvccPage page) throws IOException;
    MvccPage allocatePage(int type) throws IOException;
    long pageCount() throws IOException;
    void force() throws IOException; // pluggable fsync
    SyncPolicy syncPolicy();
    void close() throws IOException;

    enum SyncPolicy { FULL, METADATA_ONLY, NONE }
}
// MvccPage: ByteBuffer (2GB ceiling, GC) → MemorySegment+Arena (JDK 22, 64-bit, deterministic)
```

Implementation	Backing store	force()	Source	Status
FileChannelPageVolume	FileChannel + ReentrantLock	channel.force(FULL/META)	MapDB FileChannelVol Apache 2.0	NOW
OffHeapPageVolume	MemorySegment[] array (O(1) by page id)	no-op (NONE)	MapDB MemoryVol pattern Apache 2.0	NEXT
MappedPageVolume	FileChannel.map() chunked MemorySegment+Arena	segment.force() per chunk	MapDB MappedFileVol Apache 2.0	NEXT
FaultInjectingPageVolume	Wraps any above FaultSchedule injection	configurable failure	Original — crash proofs	NEXT
ODirectPageVolume	FileChannel+ ExtendedOpenOption.DIRECT	none (bypass page cache)	HerdDB ODirectFileOutput Apache 2.0	Stream FUTURE
IoUringPageVolume	FFM Linker → liburing io_uring SQ/CQ rings	IORING_OP_FSYNC async	JEP 454 JDK 22+ Jasny PVLDB 2026	FUTURE

PART 5

Optimizer — L4 Cost Model (Not a Preprocessing Layer)

Verified from OptimizerImpl.java · DelosNativeTableCostLookup.java · Devil's Advocate analysis

The definitive conclusion from comparing PostgreSQL, MariaDB, H2, Apache Calcite, Volcano/Cascades (Graefe 1995), and ORCA (Soliman SIGMOD 2014): a preprocessing transform layer between bind() and optimizeStatement() is wrong for Derby because (a) Derby's preprocess() runs inside optimizeStatement() — the tree before that call is not stable, (b) Derby already has ConstantExpressionVisitor and the Visitor pattern, (c) Derby's nodes couple logical, physical, and code-generation state in a way that makes safe tree rewrites unsafe without knowing Derby's preprocessing invariants. Two SQL frontends is incoherent for mixed JOIN queries. The correct intervention is the one already partially built: L4.

What L4 is

Derby's OptimizerImpl already calls (verified in FromBaseTable.java):

```
DelosNativeTableCostLookup.observeIfEnabled(tableDescriptor, costEstimate)
tableDescriptor.getStorageProviderName() // "delos_mvcc" or "default"
```

The gap: observeIfEnabled() currently reads cost estimates but does not write them back into Derby's StoreCostController.getScanCost() return value.

L4 fix: wire EngineMvccTableAccess.estimateTableCost() into the cost value that StoreCostController reports upward to OptimizerImpl.costPermutation().

Result: for a 1M-row delos_mvcc table with WHERE id=42, OptimizerImpl correctly picks index scan (cost ~1) over full scan (cost ~1M). Today it guesses from Derby's generic page-count formula.

Infrastructure already built (H1-H4):

```
DelosCostTableTableAccess · DelosTableCostEstimate · DelosStoreCostTuning
EngineMvccTableAccess.estimateTableCost() · VersionedStorageExecutionBridge.stats()
```

What we explicitly decided NOT to build for optimization**x No preprocessing transform layer between bind() and optimizeStatement()**

Derby's preprocess() runs inside optimizeStatement(). Operating before it means operating on an incompletely preprocessed tree (pre-CNF normalization, pre-PredicateList population, pre-FromSubquery replacement). Verified from DMLStatementNode.optimizeStatement() source.

X No HEP-style rule engine cloned from Calcite against Derby's nodes

Calcite's HEP works on immutable RelNode. Derby's QueryTreeNode is mutable. A 'rule engine' over mutable Derby nodes is the Visitor pattern with extra naming. Derby already has Visitor. ConstantExpressionVisitor already does constant folding.

X No Volcano/Cascades implementation

Cascades requires immutable plan nodes and equivalence sets (memo). Derby's 131 mutable node types were not designed for this. Retrofitting them would require a complete Derby compiler rewrite.

X No two SQL frontends

A JOIN between heap and delos_mvcc tables produces one QueryTreeNode tree, one OptimizerImpl call, one generated activation class. Splitting at SQL level is incoherent. ORCA (Soliman 2014) replaces the optimizer, not the parser — and even that requires a complete plan-translation layer.

PART 6**Performance Improvements — Single Frontend, Zero Compatibility Co**

All improvements localized to delos_mvcc execution path

NOW**P-1****Batch fetch inside DelosTableScanResultSet**

Fetch N rows from EngineMvccTableAccess.scan() per getNextRowCore() call. Serve
~~one-by-one upward to Derby's Volcano interface. Cache locality for range scans~~

Source: Derby scan interface

NOW**P-2****Compound filter before ExecRow allocation**

Evaluate all equality predicates as a compound filter before allocating ExecRow. Avoids
~~allocation for rows rejected by selective filters~~

Source: Derby execution

NEXT**P-3****ART for DelosMvccIndex.candidatesByKey**

Replace TreeMap<Object, LinkedHashSet<K>> with Adaptive Radix Tree. 2–4x OLTP index
~~throughput. ART O(1) vs B-tree O(log n) better cache utilization~~

Source: Leis et al. ICDE 2013

NOW**P-4****ExecRow template reuse in materialize()**

Reuse fixed ExecRow per scan via resultRowTemplate. Eliminates major GC pressure source
~~on large scans. Lowest risk highest value change~~

Source: Derby execution

NEXT**P-5****L4 optimizer cost consumption**

Wire EngineMvccTableAccess.estimateTableCost() into StoreCostController. Correct
~~scopes path selection for delos_mvcc (index vs full scan). H1–H4 already built~~

Source: H1–H4 infrastructure

NEXT**P-6****Group commit for MvccTransactionOutcomeLog**

Batch outcome log entries, single fsync per group. At N concurrent writers: N× throughput
~~improvement. MvccTransactionOutcomeLog.appendFsyncBoundary() is the exam~~

Source: LeanStore VLDB 2024

FUTURE**P-7****Parallel morsel scan for OLAP (guarded property)**

Partition key set into N morsels, assign to virtual thread pool (JDK 21). Each morsel carries
own MvccCapturedVisibility copy. Guarded by session property. Research-mode feature for

Source: Leis et al. SIGMOD 2014

PART 7

Complete Implementation Sequence — 25 Items

Ordered by dependency and risk

#	Item	Layer	Weeks	Status
I-1	ReentrantLock in MvccPageFile (7 synchronized methods) Eliminates virtual thread pinning. MapDB ByteBufferVol.growLock pattern (Apache 2.0).	I/O	<1w	NOW
I-2	Remove lockAcquired from DelosMutationPreparation Field always false, enforced by constructor throw. Dead API surface.	Contract	<1d	NOW
I-3	AND-conjunction fix in DelosTableScanResultSet if (term>0 && length>1) → if (length>1). Verified from PredicateList.java:3016.	SQL	<1d	NOW
I-4	ExecRow template reuse in materialize() (P-4) Reuse resultRowTemplate per scan. Largest zero-risk GC win.	SQL	<1d	NOW
I-5	Batch fetch in DelosTableScanResultSet (P-1) Internal N-row buffer per getNextRowCore(). Tunable via DelosStoreCostTuning.	SQL	3d	NOW
M-1	MvccVersionChain: synchronized → ReentrantReadWriteLock Row-granularity read/write lock. Use MvccCapturedVisibility for all visibility checks.	MVCC	1w	NEXT
M-2	MvccTransactionRegistry: volatile TransactionSnapshot publication Lock-free read path for statusOf() / commitSequenceOf(). Published on each commit/abort.	MVCC	1w	NEXT
M-3	MvccVersionStore interface + InMemoryVersionStore 5-method interface. ConcurrentHashMap<K, List<MvccVersion<V>>> backing. Replaces chain-in-table.	MVCC	2w	NEXT
M-4	MvccPageFile as current-version store Writes go to MvccPageFile. Updates push old version to MvccVersionStore first.	MVCC	2w	NEXT
M-5	MvccLazyScan cursor Key snapshot at open. Per-step: check MvccPageFile → MvccVersionStore. No table lock between steps.	MVCC	1w	NEXT
M-6	Recovery: MvccTransactionRegistry rehydration Read MvccTransactionOutcomeLog on open. Rebuild registry. Apply MvccPageMutationLog.	MVCC	2w	NEXT
M-7	Index update integration with Main+Delta write path DelosMvccIndex.addCandidate/removeCandidate called from write path, not version chain.	MVCC	1w	NEXT
M-8	MvccVacuumEngine: vacuum MvccVersionStore MvccVacuum.vacuum() targets MvccVersionStore. MvccPageFile never touched by vacuum.	MVCC	1w	NEXT
N-1	DelosPageVolume interface + 4 implementations FileChannel, OffHeap, FaultInjecting, Mapped. Wire into MvccPageFile callers.	I/O	2w	NEXT
N-2	MvccPage: ByteBuffer → MemorySegment + Arena (JDK 22) Removes 2GB ceiling. Deterministic off-heap deallocation. FileChannel.map() + Arena.	I/O	1w	NEXT
N-3	DelosVolumeFactory matched-pair pattern Consistent backing mode for page + mutation-log + outcome-log + index per provider.	I/O	1w	NEXT
L-4	L4 optimizer cost consumption Wire EngineMvccTableAccess.estimateTableCost() into StoreCostController. H1–H4 seam exists.	SQL	2w	NEXT
P-3	ART for DelosMvccIndex.candidatesByKey Replace TreeMap with Adaptive Radix Tree. 2–4x OLTP index throughput.	MVCC	2w	NEXT
P-6	Group commit for MvccTransactionOutcomeLog Batch fsync. appendFsyncBoundary() is the seam. High-concurrency throughput multiplier.	I/O	2w	NEXT
K-1	K1 heap feasibility probe Can EngineHeapTableAccess.scan() honestly delegate through Delos contract? Binary A/B answer.	-	1w	FUTURE
F-1	ODirectPageVolume HerdDB ODirectFileOutputStream (Apache 2.0). Bypass OS page cache. JDK 10+.	I/O	1w	FUTURE
F-2	Graveyard Index Move tombstones out of main index scan path. Alhomssi & Leis VLDB 2023 Section 4.1.	MVCC	6w	FUTURE

#	Item	Layer	Weeks	Status
F-3	Adaptive version storage Full-copy vs delta per tuple hot/cold. <code>deadVersionEstimate()</code> already tracks trigger.	MVCC	6w	FUTURE
F-4	Scalable commit protocol Autonomous commit. Avoids write-set touch on commit. Alhomssi & Leis VLDB 2023 Section 3.	MVCC	5w	FUTURE
F-5	IoUringPageVolume FFM Linker + liburing. Async I/O. JDK 22+. Only after FileChannel proven as bottleneck.	I/O	5w	FUTURE

PART 8

Explicit Non-Goals — What DelosDB Will Not Build

Each with evidence and rationale

✗ PostgreSQL-style MVCC (O2N append-only)

Full-row copy on every update. Table bloat — dead tuples in same pages as live data. O2N chain: every index lookup starts at oldest tuple, traverses to newest. Pavlo (CMU 2023): 'if building new MVCC DBMS today, should NOT do it the way PostgreSQL does.'

✗ synchronized anywhere in the MVCC or I/O path

JEP 444 (JDK 21): synchronized pins virtual thread carrier. JEP 491 (JDK 24): fixes most cases but not native calls — and `channel.force(true)` is a native syscall. `ReentrantLock` solves all cases on all JDK versions.

✗ Full materialization scan (current `MvccTable.openScan()`)

Entire table locked and copied to a List before first row returned. 100K-row scan blocks all concurrent writers for its duration. `MvccLazyScan` (M-5) replaces this.

✗ DelosStorageDispatch before two live providers exist

PostgreSQL `smgrsw[]` carried one entry and 'we only have `md.c` at present' for decades. Build the dispatch table only after K1 confirms heap can honestly be a live provider.

✗ io_uring by default

Jasny et al. PVLDB 2026: `io_uring` benefits are workload-dependent — high concurrent IOPS with batched submission. A synchronized `FileChannel` with one I/O per commit is not that workload. `IoUringPageVolume` is a named future slot gated on benchmark evidence.

✗ MariaDB 218-method handler interface

Verified from `sql/handler.h`. No common VFS across engines. InnoDB uses `os0file`, MyISAM uses `mysys`, RocksDB uses RocksDB Env. Each engine owns its I/O. `DelosPageVolume` is MVCC-internal — the heap adapter uses Derby's `StorageFactory`.

✗ Calcite rule catalog import (43,377 LOC, 157 files)

All 157 rules reference `LogicalFilter`, `LogicalProject`, `EnumerableTableScan`. None of Derby's 131 `QueryTreeNode` types are those types. No rule fires against Derby's plan nodes without a complete Derby compiler rewrite.

✗ Preprocessing transform layer between `bind()` and `optimizeStatement()`

Derby's `preprocess()` runs inside `optimizeStatement()`. The tree before that call has unnormalized WHERE, unpopulated `PredicateList`, unflattened `FromSubquery`. Derby already has `ConstantExpressionVisitor`. The Visitor pattern already exists. Verified from `DMLStatementNode.optimizeStatement()` source.

R

References

All sources verified from primary material

- [1] Wu, Y., Arulraj, J., Lin, J., Xian, R. & Pavlo, A. (2017). An Empirical Evaluation of In-Memory Multi-Version Concurrency Control. PVLDB 10(7), 781–792. Definitive survey: four MVCC design axes, system-by-system matrix.
- [2] Alhomssi, A. & Leis, V. (2023). Scalable and Robust Snapshot Isolation for High-Performance Storage Engines. PVLDB 16(6), 1426–1438. Graveyard Index, adaptive version storage, scalable commit protocol, N2O ordering.
- [3] Leis, V. (2024). LeanStore: A High-Performance Storage Engine for NVMe SSDs. PVLDB 17(12). Unified I/O abstraction, version chain architecture, group commit.
- [4] Jasny, M., El-Hindi, M., Ziegler, T., Leis, V. & Binnig, C. (2026). io_uring for High-Performance DBMSs: When and How to Use It. PVLDB 19(1). io_uring benefits are workload-dependent — not default for DelosDB.
- [5] Graefe, G. (1995). The Cascades Framework for Query Optimization. IEEE Data Eng. Bull. 18(3), 19–29. Transformation rules + implementation rules, memoization. Requires immutable plan nodes — not applicable to Derby's mutable AST.
- [6] Soliman, M.A. et al. (2014). Orca: A Modular Query Optimizer Architecture for Big Data. SIGMOD 2014, 337–348. Portable optimizer via metadata abstraction. Replaces optimizer, not parser. One SQL frontend.
- [7] Leis, V., Kemper, A. & Neumann, T. (2013). The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. ICDE 2013. 2–4x over B-tree/red-black for OLTP. Replacement for DelosMvccIndex.candidatesByKey TreeMap (P-3).
- [8] Leis, V. et al. (2014). Morsel-Driven Parallelism: A NUMA-Aware Query Evaluation Framework for the Many-Core Age. SIGMOD 2014. Morsels assigned to virtual thread pool. P-7 guarded OLAP scan.
- [9] Pavlo, A. (2023). The Part of PostgreSQL We Hate the Most. CMU Blog. O2N chain, full-row copy, table bloat, physical index pointers. Why not PostgreSQL MVCC.
- [10] OpenJDK JEP 444: Virtual Threads (Java 21). <https://openjdk.org/jeps/444> synchronized pins carrier thread during I/O.
- [11] OpenJDK JEP 491: Synchronize Virtual Threads without Pinning (Java 24). <https://openjdk.org/jeps/491> — does not unpin native syscalls (channel.force).
- [12] OpenJDK JEP 454: Foreign Function & Memory API (Java 22, finalized). <https://openjdk.org/jeps/454> MemorySegment, Arena, FileChannel.map() + Arena.
- [13] HerdDB source: DataStorageManager, ManagedFile, ODirectFileOutputStream. License: Apache 2.0. <https://github.com/diennea/herddb>
- [14] MapDB source: Volume, ByteBufferVol, MappedFileVol, MemoryVol, Factory. License: Apache 2.0. <https://github.com/jankotek/mapdb>
- [15] Apache Calcite 1.42.0 source: Table, FilterableTable, ProjectableFilterableTable, HepPlanner, VolcanoPlanner, CsvFilterableTable. License: Apache 2.0. <https://github.com/apache/calcite>
- [16] PostgreSQL source: optimizer/plan/planner.c, optimizer/prep/prepjointree.c, optimizer/prep/prepqual.c, rewrite/rewriteHandler.c, storage/smgr/smgr.c. <https://github.com/postgres/postgres>
- [17] MariaDB source: sql/sql_select.cc, sql/opt_subselect.cc, storage/innobase/row/row0vers.cc, storage/innobase/trx/trx0purge.cc. <https://github.com/MariaDB/server>
- [18] H2 Database source: command/query/Optimizer.java, command/query/Select.java, expression/Expression.java, expression/condition/ConditionAndOr.java. License: MPL 2.0. <https://github.com/h2database/h2database>
- [19] Apache Derby / DelosDB source: delosdb-storage-mvcc, delosdb-engine-kernel, delosdb-engine, delosdb-storage-derby. License: Apache 2.0. <https://github.com/ggeorg/delosdb>